Mara Olvera Angel
David
3CM31

"Estructura de datos"

(Pila, Cola, Lista mediante arreglos).

1- Pila: Se creo una clase pila utilizando un arreglo de 100 posiciones para almacenar numeros enteros. El programa permite insertar elementos, extraerlos, mostrar todos los valores almacenados y consultar la cantidad de elementos. Su funcionamiento se basa en el principio LIFO (ultimo que entra, primero que sale), por lo que los elementos se agregan y eliminan desde el inicio del arreglo.

→ Errores encontrados y soluciones: Durante su elaboración se presentarón problemas al insertar elementos cuando la pila estaba llena y al intentar extraer valores cuando estaba vacia. Tambien fue necesario controlar las posiciones del arreglo. Estos problemas se solucionaron implementando condiciones de validación y excepciones para evitar operaciones incorrectas.

2- Cola: Se desarrollo una clase llamada cola, que utiliza un arreglo de 100 posiciones para almacenar numeros enteros. Al igual que el programa de pila, este cuenta con funciones para insertar elementos al final, extraerlos desde el inicio, mostrar su contenido y consultar la cantidad de elementos. Su funcionamiento sigue el principio FIFO (primero en entrar, primero en salir).

→ Errores encontrados y soluciones: Se presentaron dificultades al mantener el orden correcto de los elementos al insertarlos y extraerlos. De igual forma, se consideraron los casos en los que la cola estuviera vacia o alcanzara su capacidad maxima. Para darle solución al problema, se ajustaron las posiciones de inserción y extracción mediante ciclos y se agregarón validaciones con excepciones.

3- Lista: Se implemento una clase llamada Lista, utilizando un arreglo de 100 posiciones para almacenar datos enteros. A diferencia de la pila y cola, permite insertar y extraer elementos en cualquier

posición disponible. También cuenta con funciones para mostrar los elementos almacenados y consultar la cantidad de los datos.

→ Errores encontrados y soluciones: En este caso, se identificaron errores al insertar y extraer elementos en posiciones incorrectas. así como al recorrer y desplazar los valores dentro del arreglo. Se solucionarón implementando validaciones para las posiciones y ciclos que permiten recorrer el arreglo y acomodar los elementos sin perder información.

→ Conclusión general (arreglos): Los tres codigos permiten trabajar con diferentes estructuras de datos en C++, utilizando programación orientada a objetos mediante clases y arreglos de tamaño fijo. La principal diferencia entre ellos es la manera en que organizan y manipulan los elementos: la pila trabaja con el principio LIFO, la cola utiliza FIFO y la lista permite insertar y eliminar elementos en distintas posiciones.

Aunque los tres utilizan arreglos para almacenar información, cada uno modifica y recorre sus posiciones de manera distinta.

Maca olvera Angel
David
3CM31

"Estructura de datos"

(Pila, cola y Lista mediante apuntadores).

→ **Pila:** En este caso se construyo una pila utilizando nodos y apuntadores, donde cada elemento se concecta con el siguiente. Se implementaron funciones para insertar elementos en la cima, extraerlos, consultar el elemento superior, mostrar el contenido y conocer la cantidad de datos. Tambien incluye un menu independiente para controlar las operaciones de la pila.

→ **Errores encontrados y soluciones:** Se tuvieron que controlar los apuntadores al insertar y extraer elementos, especialmente cuando la pila se encontraba vacia. Tambien era necesario liberar correctamente la memoria de los nodos eliminados. Estos detalles se solucionaron mediante condiciones de validación, apuntadores auxiliares y el uso de "delete" para evitar ocupar memoria innecesariamente.

→ **Cola:** En este programa se implemento una cola mediante una lista de nodos enlazados y apuntadores. Cada nodo almacena un numero entero y una referencia al siguiente nodo. Cuenta con operaciones para insertar elementos al final, extraerlos desde el frente, consoltar el primer elemento, mostrar el contenido y vacir la cola. Al igual que los demas, incluye un menú para facilitar su manejo.

→ **Errores encontrados y soluciones:** Uno de los principales problemas fue mantener correctamente los apuntadores del frente y del final al insertar y eliminar los elementos. Tambien se debia evitar que quedaran apuntadores incorrectos cuando la cola se vaciaba. Esto se soluciono actualizando las diferencias y referencias entre nodos y liberando la memoria con "delete" despues de cada extracción.

→ **Lista:** Se desarrollo una lista enlazada simple utilizando apuntadores para conectar sus diferentes nodos. Cada nodo contiene un dato y una referencia al siguiente. El programa permite insertar elementos al inicio y al final, eliminar valores especificos, buscar elementos,

recorrer la lista y consultar su cantidad. Además, cuenta con una opción para liberar todos los nodos.

→ Errores encontrados y soluciones: Durantes su desarrollo surgieron dificultades al modificar los enlaces entre nodos, principalmente, al eliminar al inicio o enmedio de la lista. Para resolverlo, se actualizaron los apuntadores correspondientes y se utilizaron condiciones para comprobar si los elementos existían antes de eliminarlos.

→ Conclusión general (apuntadores): Mediante estas practicas aprendimos a implementar las estructuras de pila, cola y lista en C++ utilizando POO y apuntadores. A diferencia de los arreglos, los apuntadores permiten crear nodos dinamicamente y conectar los elementos mediante referencias, sin establecer una capacidad fija de almacenamiento. La diferencia en las estructuras es claramente la organización que ocupan. Además, comprendimos la importancia de administrar correctamente la memoria dinamica para evitar errores durante la ejecución de los programas.

Maca olvera Angel
David
JCM31

"Estructura de datos"

(Pila, Cola y Lista mediante librerías).

1. Pila: Se implemento una pila utilizando la libreria <stack> de C++, la cual permitio almacenar y administrar elementos mediante funciones ya establecidas. Se utilizaron operaciones como Push() para insertar datos, pop() para extraerlos, top() para consultar el elemento superior y empty() para comprobar si la pila contenia elementos.

→ Errores encontrados y soluciones: Durante la programación se presentaron dificultades al intentar consultar o extraer elementos de una pila vacia. Para solucionarlo se agregaron comprobaciones con la función "empty()" antes de realizar dichas operaciones, evitando errores durante su ejecución.

2. Cola: Se trabajo con la libreria <queue> para realizar una cola que permitio almacenar elementos respetando su orden de llegada. Se utilizaron funciones como push() para agregar datos, pop() para retirarlos, front() para consultar el primer elemento y size() para conocer la cantidad almacenada.

→ Errores encontrados y soluciones: Se presentaron algunos problemas al extraer elementos cuando la cola estaba vacia y al consultar el orden de los datos. Estos se solucionaron mediante validaciones con empty() y utilizando correctamente las funciones de la libreria para mantener el funcionamiento FIFO.

3. Lista: Se desarrollo una lista utilizando la libreria <list>, con la cual fue posible almacenar y organizar diferentes elemeltos. Se implementaron operaciones para insertar datos al inicio y al final, eliminar elementos, recorrer la lista y consultar su tamaño mediante funciones como push_front(), push_back(), remove() y size().

Mora Olvera Angel David 3CM31

-> Errores encontrados y soluciones: Durante su elaboración se tuvieron dificultades al recorrer los elementos y utilizar correctamente las funciones de inserción y eliminación. Se solucionaron utilizando iteradores y verificando las operaciones realizadas para mantener un orden correcto dentro de la lista.

-> Conclusión general (librerías): Con estas practicas logramos implementar las estructuras de pila, cola y lista mediante liberías propias de c++. A diferencia de los ejercicios anteriores con arreglos y apuntadores, en esta ocasión utilizamos funciones previamente definidas que facilitaron el manejo y almacenamiento de los datos. Esto nos permitio comparar las distintas formas de trabajar con estructuras de datos y comprender las ventajas de utilizar las herramientas que ofrece c++.

Mara olvera Angel
David
3CM31

"Conclusiones"

(codigos – 1er Parcial).

1.- Fundamentos de programación (Hola mundo): Se creo la clase "Saludo", incorporando un metodo publico encargado de mostrar un mensaje en pantalla mediante std::cout. Se utilizo un metodo para llamar un objeto y ejecutar el programa.

2.- Suma de dos numeros: Se diseño la clase "operación", con un metodo que recibe dos valores como parametro y devuelve el resultado de su suma. Se creo una instancia para enviar los valores y obtener el resultado.

3.- Calculadora basica: Se desarrollo la clase "CalculadoraBasica", integrando metodos para realizar las operaciones aritmeticas fundamentales: suma, resta, multiplicación y división. Se utilizaron objetos para ejecutar cada operación.

4.- Sobrecarga: se implemento la sobrecarga de funciones, utilizando metodos con el mismo nombre, pero con diferentes cantidades de parametros. Esto permitio realizar operaciones dependiendo de los argumentos recibidos.

5.- Herencia: Se creó una nueva clase a partir de la calculadora anteriors, heredando sos atributos y metodos. De esta manera se reutilizo el codigo existente y se agregarón nuevas funcionalidades.

6.- Herencia y sobreescritura: Se amplio la clase de la calculadora mediante la sobreescritura de metodos heredados. se implementaron operaciones como multiplicación, potencia y dNisión mediante calculos sucesivos.

7.- Creación de datos: Se definierón nuevos tipos de datos mediante estructuras y clases. Se desarrollaron registros para almacenar informacióñ de automoviles, como precio y año y datos personalrs como nombre, apellidos, genero y edad.

7.- bis. Identificar tamaños: Se utilizaron funciones para conocer el espacio que ocupan los diferentes tipos de datos basicos y los creados mediante estructoras y clases, empleando "sizeof()".

8.- Promedio: Se elaboro un programa que recibe 5 numeros y realiza diferentes operaciones matematicas. Se implementaron calculos para obtener la suma, promedio, media, valor max. y min.

9.- Promedio con arreglo: Se modifico el ejercicio anterior para almacenar los 5 valores dentro de un arreglo. Posteriormente, se recorrieron sus posiciones para realizar los calculos correspondientes.

10.- Promedio con apuntador al arreglo: Se utilizaron apuntadores para acceder indirectamente a los valores almacenados en un arreglo. A traves de ellos se realizaron las operaciones de suma, resta, promedio, max. y min.

11.- Matrices: Se trabajo con matrices de tipo A y B, implementando operaciones para multiplicar una matriz por una constante y realizar la multiplicación entre ambas matrices.

12.- Nuevo tipo de dato como arreglo: Se retomaron los tipos de datos creados en el ejercicio 7 para formar arreglos de personas y carros. Esto permitio almacenar varios registros utilizando estructuras y clases.

13.- Nuevo tipo de dato con apuntador al arreglo: Se utilizaron apuntadores para acceder indirectamente a los elementos de un arreglo que contiene información de personas, permitiendo consultar sus datos mediante referencias.

14.- Recursividad: Se desarrollaron funciones recursivas para resolver problemas matematicos como el calculo del factorial y la sucesión de Fibonacci.

15.- Recursividad - Fractales: Se aplico la recursividad para generar figuras fractales mediante divisiones sucesivas. Se trabajo con representaciones del triangulo de Sierpinski y el polvo de Cantor.